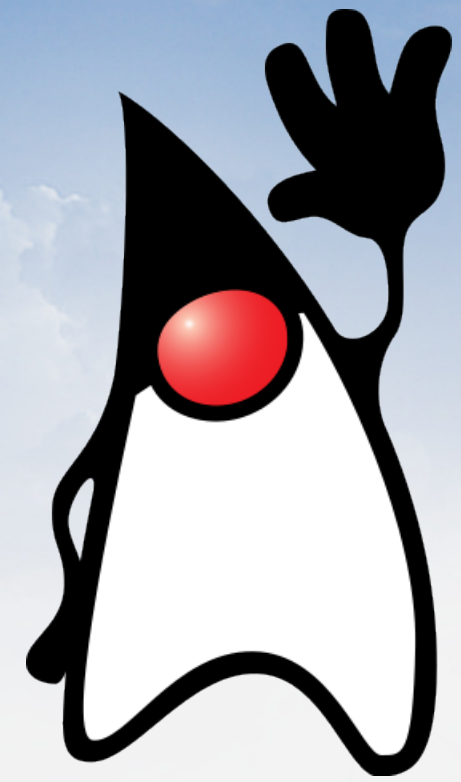




Memory

The Stack and Heap

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

overriding equals

```
public class Person {  
    private String name;  
    private Calculator c;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
        c=null;  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(age, c, name);  
    }  
    // The overridden code is put right  
    // because of the space  
}
```

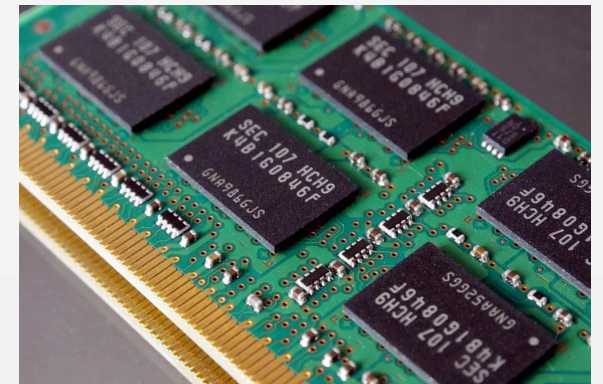
Objects.equals() considers two nulls to be "equal". You still need to override equals for objects though.

```
@Override  
public boolean equals(Object obj) {  
    if (this == obj)  
        return true;  
    if (obj == null)  
        return false;  
    if (getClass() != obj.getClass())  
        return false;  
    Person other = (Person) obj;  
    return age == other.age &&  
        Objects.equals(c, other.c) &&  
        Objects.equals(name, other.name);  
}
```

```
@Override  
public boolean equals(Object obj) {  
    if (this == obj)  
        return true;  
    if (!(obj instanceof Person))  
        return false;  
    Person other = (Person) obj;  
    return age == other.age &&  
        Objects.equals(c, other.c) &&  
        Objects.equals(name, other.name);  
}
```

Memory Overview

- Memory is valuable to have.
 - Allows us to store values of variables.
 - Methods we define in our programs
- The OS manages the memory being utilized
- What happens if there is so much memory being utilized?
- Overhead! Causing our programs to run a bit slower than usual.



Static Memory

- Static memory is memory that is provided to us by the OS.
- It is provided at compile time.
- The amount of static memory being provided depends on a certain amount of criteria.
 - How many methods did you define?
 - How many variables did you write?
 - In simple terms, what did you put in your code file!
- Static memory cannot change in size during a program run!
- “You get what you get and you don’t get upset!”
- Static Memory is stored in a **Stack Space**.

Dynamic Memory

- Dynamic memory is memory that is provided to us by us (Yes! We are the one requesting it).
- It is provided at run time.
- The amount of dynamic memory being provided depends on a certain amount of criteria.
 - How many bytes!
 - Think about data types and their size requirements!
- Dynamic memory can change in size during a program run!
 - Bigger or Even Smaller
- All dynamic memory is stored in what we call a “*heap space*”

for C/C++

- When taking the prerequisite course, you learned how C manages dynamic memory.
 - Remember malloc() and free()
 - In the C/C++ language, you would have to manually type code involving you requesting more memory the heap space.
- In C++, there are destructor methods where you free the elements that you have used there
 - Destructor is called automatically in appropriate times
 - If you have used malloc or new, this is the place to clear them

for Java

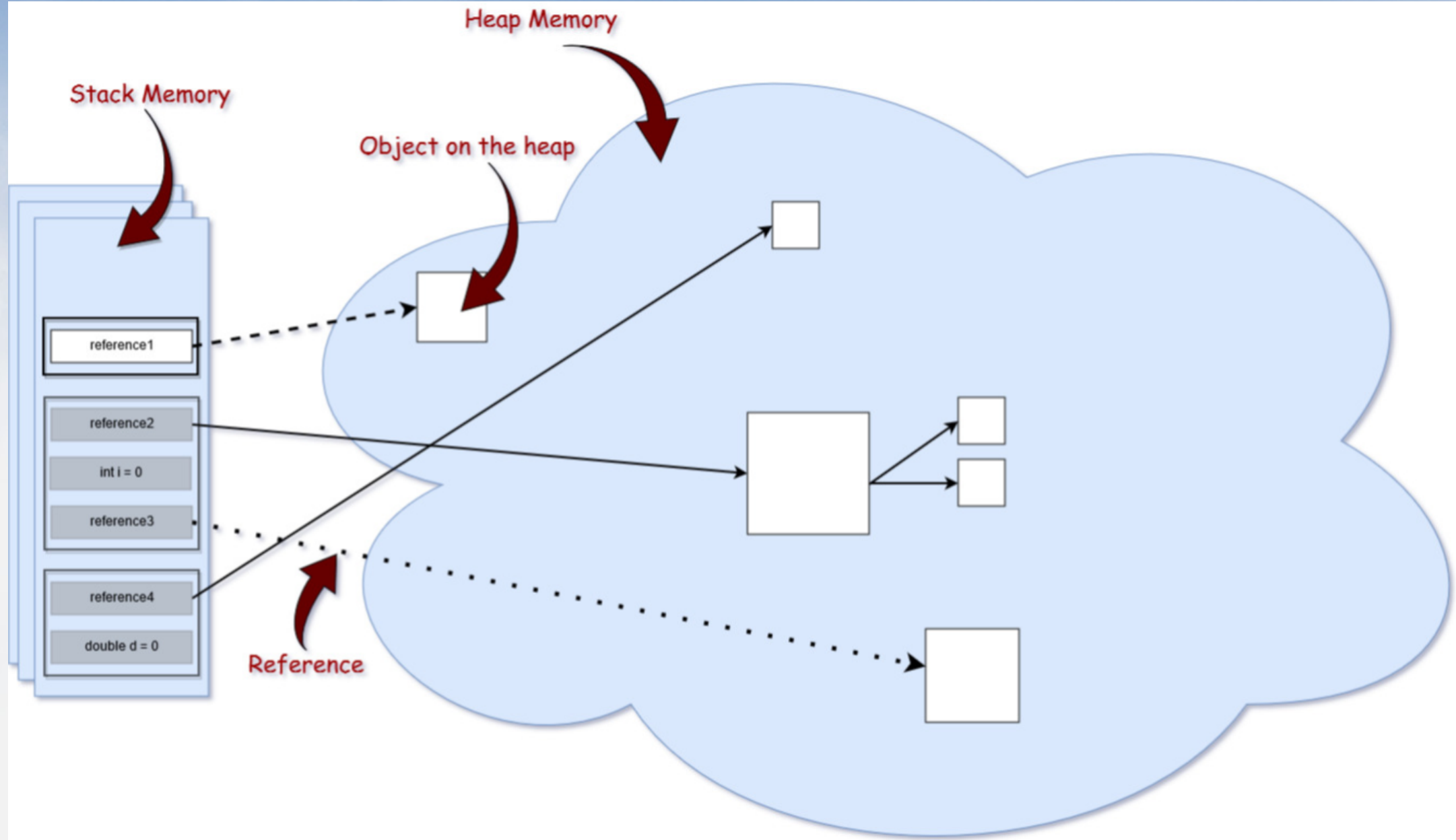
- Java Virtual machine handles the “memory freeing” job
- All automatic, you do not need to do anything.
 - Literally, you can not take part in this process directly
 - You cannot force the garbage collector to run
 - You cannot free anything!
- The system that handles memory management is called Garbage Collector
- It runs when it needs to, pauses your program, does cleaning.

Stack Space In JVM

- Java has the following primitive data types
 - byte
 - short
 - int
 - long
 - float
 - double
 - boolean
 - char
- Java places all primitive data types in the stack space.
- Java also places method definitions in the stack space.
- Java uses the LIFO (last in first out) when calling method.
 - Everytime a method is called, Java places the method call in the JVM call stack.
- If we ever overfill the stack. Java has an exception it will raise.
 - We will learn about exceptions later in the semester.

Heap Space in JVM

- So we have been discussing primitives and methods in Java.
- Java decided that all objects must be stored in the heap space.
 - So think about this...
 - Are objects basically only dynamic memory?
- Notice how the `new` works even further besides invoking some constructor.
- Everytime we invoke the constructor with *new* we are basically using `malloc()` in C.
- In other words every object stuff is pointer in Java.



The Garbage Collector

- The Garbage Collector is the process of looking at heap memory (heap space), identifying which objects are in use and which are not, and deleting the unused objects.
 - From the Oracle Website
- Each JVM may implements its own GC.
- When does the Garbage Collector work?
 - In simple terms, when we reach the end of a method/scope.
 - The garbage collector will look for any objects that are not referenced!
- We can also invoke the Garbage Collector Manually?
 - The System class has a built in method called `System.gc()`
 - DO NOT call that. In fact, the romour is that the compiler ignores or it doesn't guarantee that it will run GC

Garbage Collector

- Alive: The object/data is actively used and reachable
- Dead: The object/data is not reachable, nothing points to that.
- GC is checking for dead/alive objects from time to time, providing contiguous memory allocation for fast access/organized heap space

Main Garbage Collection Steps

Phase 1: Marking objects as alive

Garbage collector traverses all memory slots to mark the locations are alive or dead. The heap space below has dead objects colored with blue

Main Garbage Collection Steps

Phase 2: Sweeping dead objects

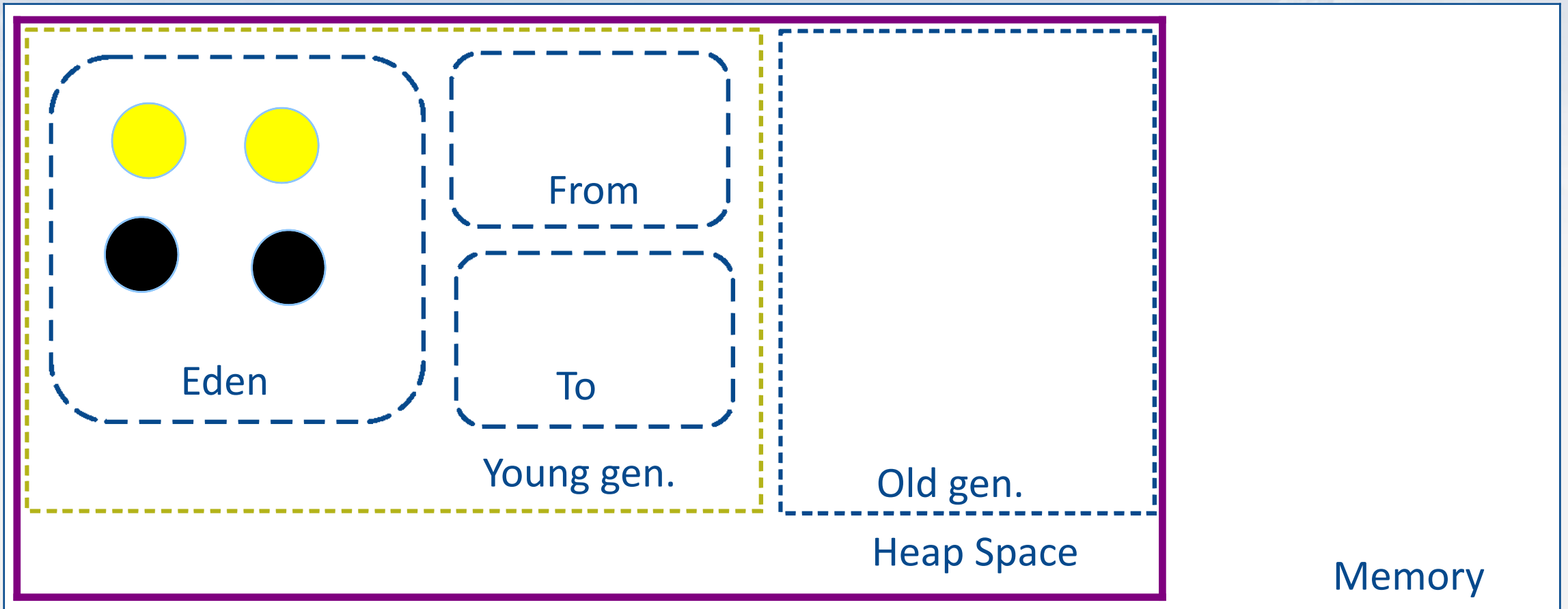
Sweep phase releases the memory fragments. It released but now there are spaces between memory fragments

Main Garbage Collection Steps

Phase 3: Compact remaining objects

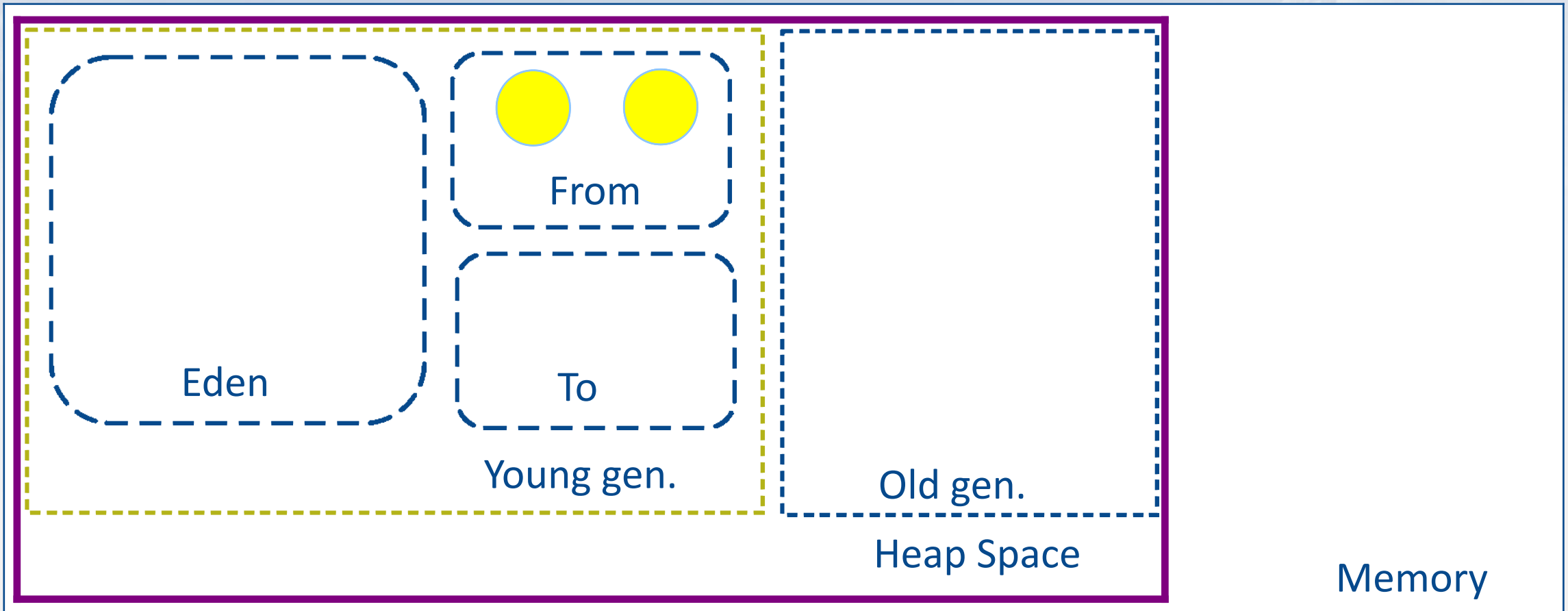
Garbage collector reorganizes the memory to provide contiguous spaces between them. Now the green locations are all available.

Yellow objects are alive,
black objects are dead.



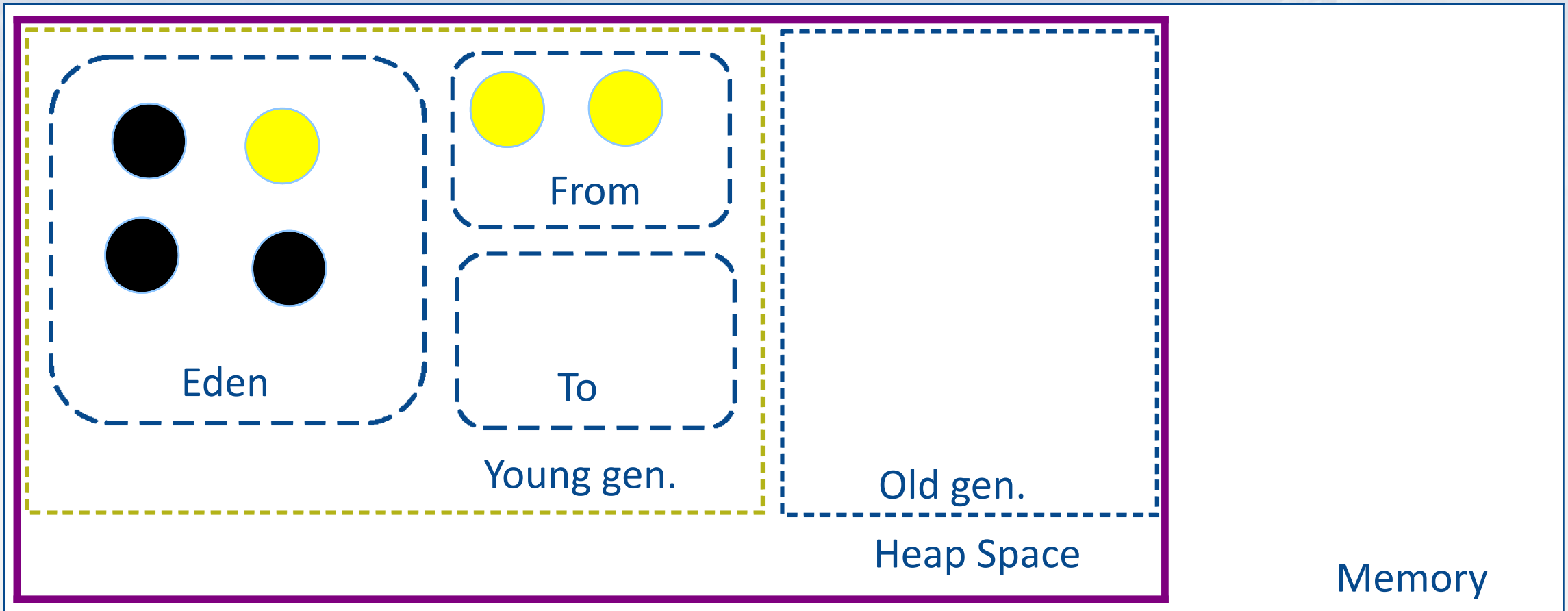
First Iteration

Yellow objects are alive,
black objects are dead.



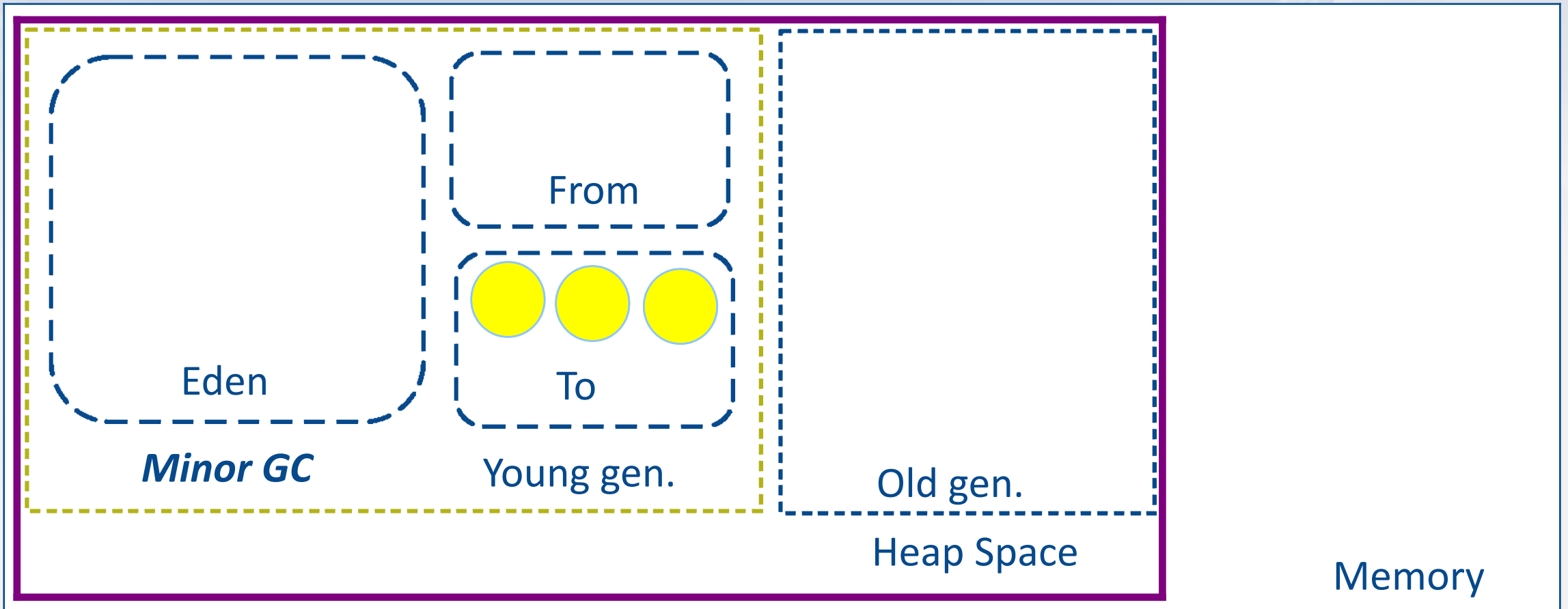
Second iteration

Yellow objects are alive,
black objects are dead.



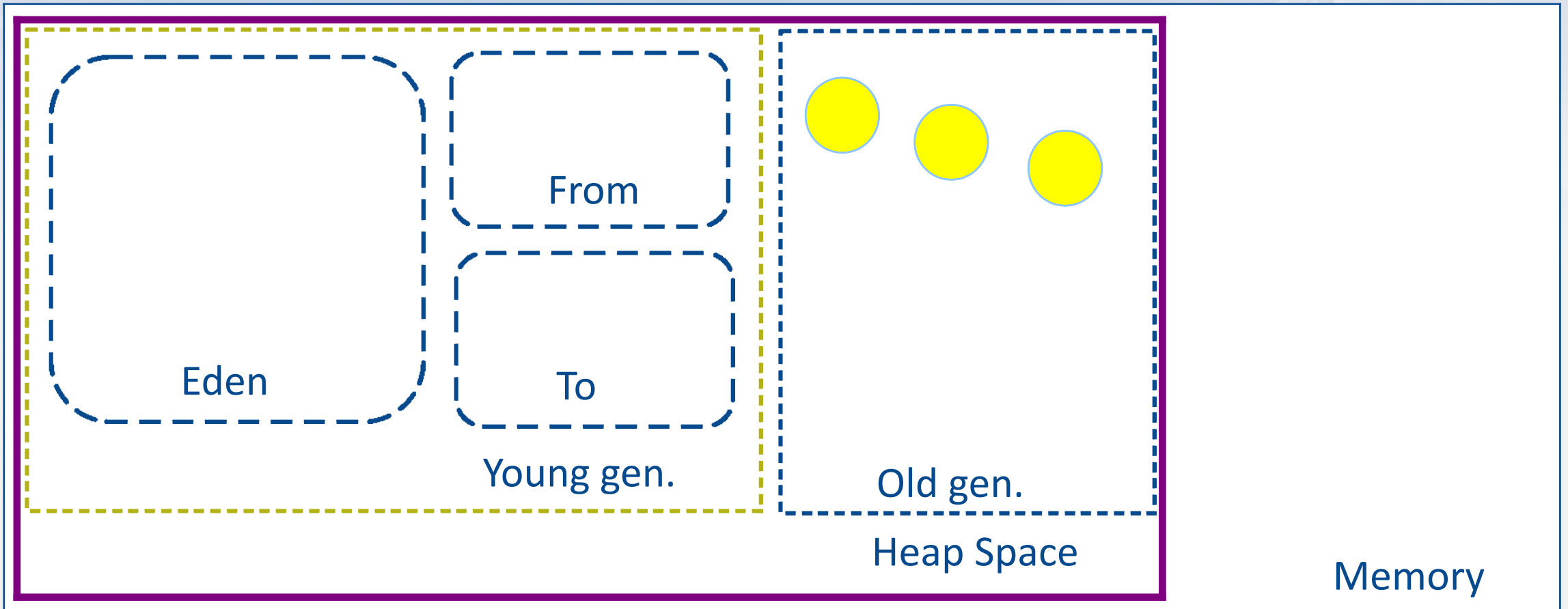
nth iteration

If some objects stay alive after multiple iterations they are put to *To* space.



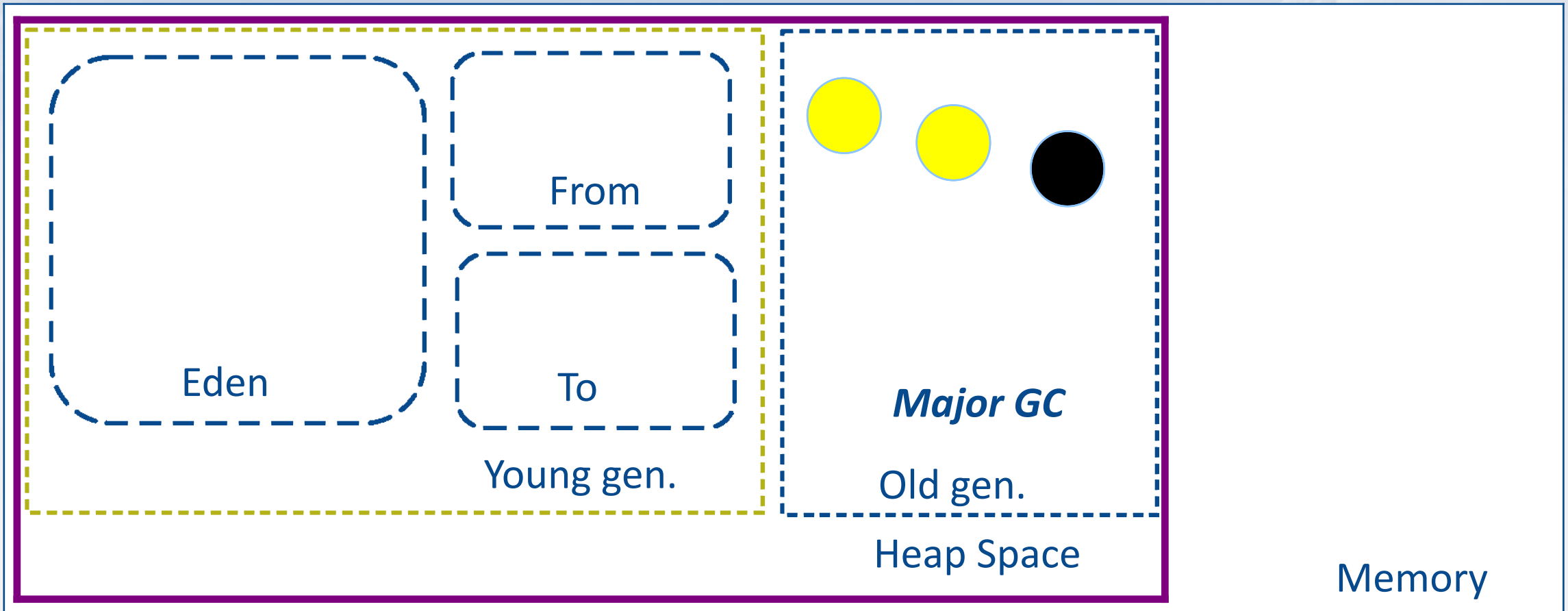
nth iteration

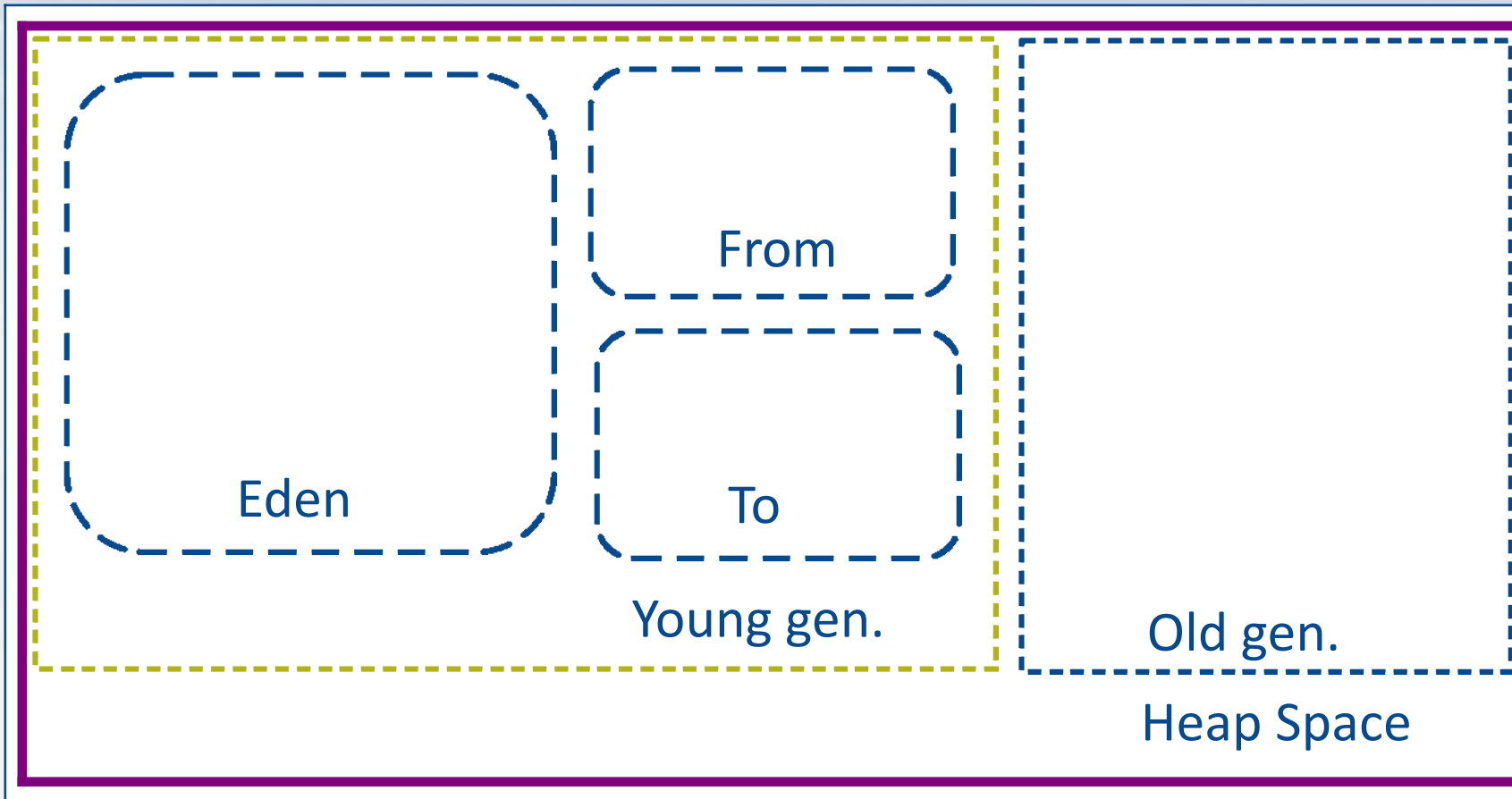
After some more iterations if objects are still alive in To space, they are moved to old gen.



nth iteration

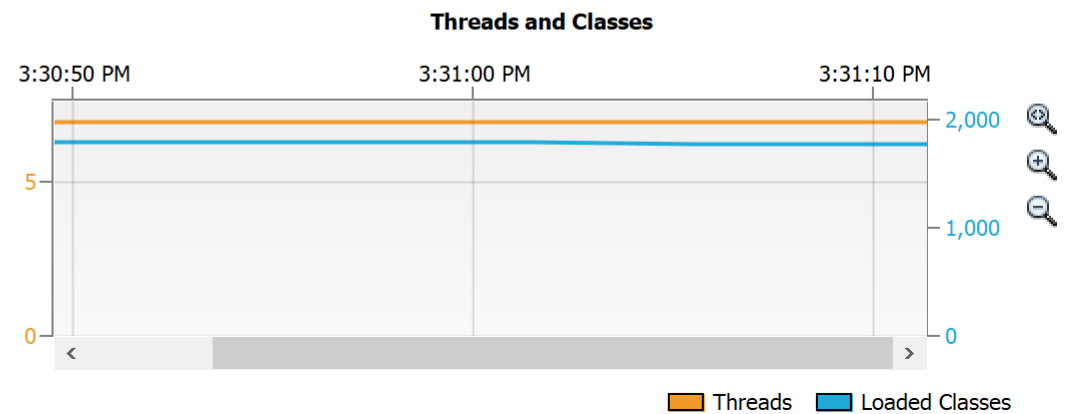
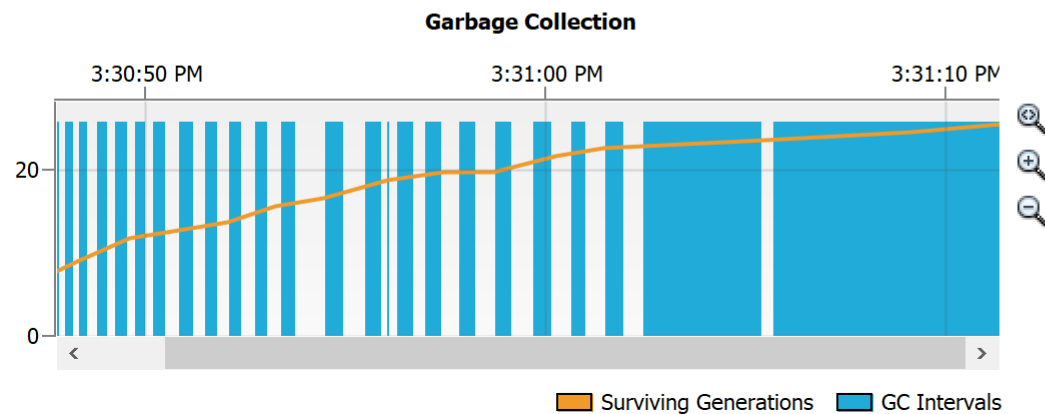
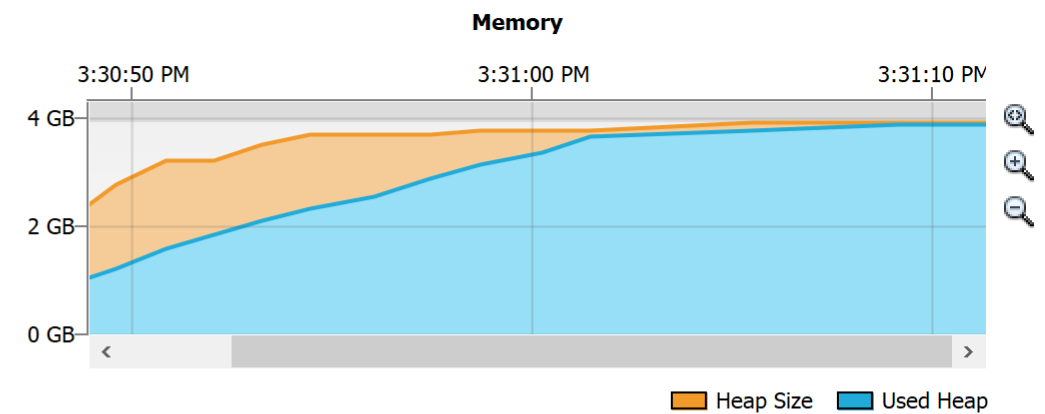
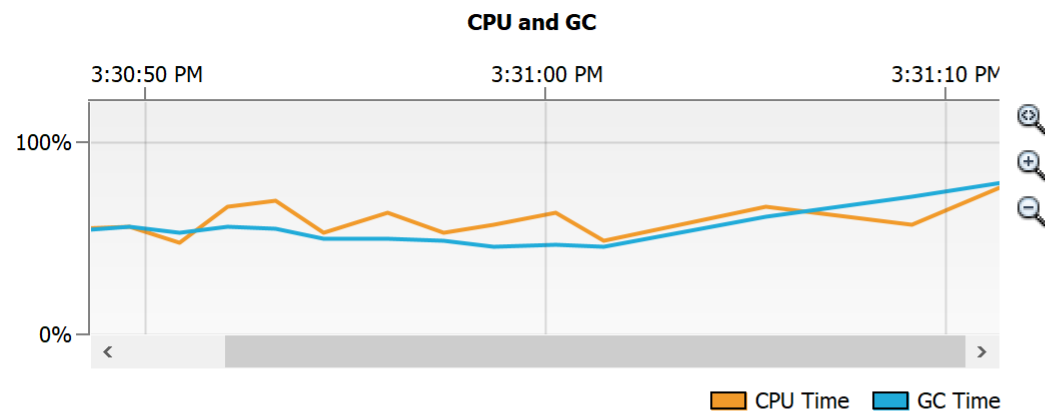
If old generation objects needs to be cleaned this is Major GC.





*Meta Space:
When the space
limit is reached,
the unused
classes and
methods are
purged.*

Memory



Can we help Garbage Collector?

- Java is optimized enough but there are a few ideas we can use.
- Use StringBuilder if you are concatenating many strings

```
public static String getNumbers(){
    StringBuilder sb=new StringBuilder();
    for(int i=0; i<5; i++){
        sb.append("" + i);
    }
    return sb.toString();
} // This is correct
```

```
public static String getNumbers(){
    String result="";
    for(int i=0; i<5; i++){
        result = result +i;
    }
    return result;
} // This is wrong
```

- Close the scanner/Files/streams when you are done.

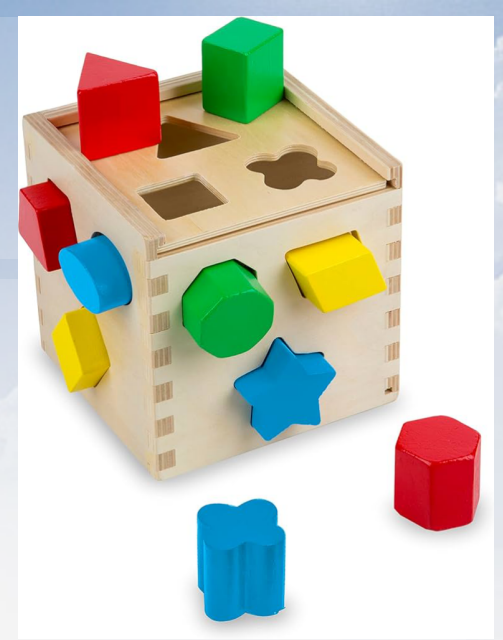
Can we help Garbage Collector?

- Nullify the objects when you do not need any more in blocks

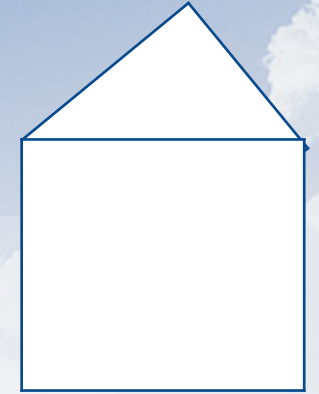
```
public static String getNumbers(){  
    ArrayList<Integer> arr=new ArrayList<>();  
    for(int i=0; i<5; i++){  
        arr.add(i);  
    }  
    String result = arr.toString();  
    arr=null; // we do not need ArrayList anymore  
    return result;  
}
```

Some exercise

- Create an abstract Shape class
 - Add `getArea()` as an abstract method
 - Add `getHeight()` as an abstract method
- Create a subclass of shape, name it as right triangle
 - Create constructor/s for that class
 - Override methods accordingly
 - add `getHypothenus()` method to that class
- Create a subclass of shape, name it as Square
 - Create constructor/s for that class
 - Override/overload methods accordingly
 - Write a method to check if a given triangle is exactly the half of this Square



Some exercise



- Create a ShapeTester class
 - Add main method inside
 - Ask the user, the type of the shape they want to create.
 - According to their answer, get the parameters for that shape from user
 - If the user enters “STOP”, stop getting the shapes from user
 - Write the shapes and areas of shapes to the screen
 - Find the total area of the shapes
 - If hypotenuse of any right angled triangle matches with a side of any square, write “You made a house”. Else do not write anything.
 - Ex: If the square’s side is 5, right angled triangle has sides 3 and 4, it will be a house

More topics to exercise

- Use upcasting, downcasting (with or without loops)
- Create an interface called `Areaable`, put `getArea()` inside that
- Implement `areaable` for all other shapes/only to the `shape` class
- Add a counter to `shape` so that when a new shape is created, the counter is incremented by one
- Try to add a static method inside one of the shapes or even inside the `shape`
- Use that static method in one of the methods/constructor